



Museo Virtual Configurable - Backend

Sistema completo de backend para el Museo Virtual 3D. Persiste imagenes, audio, configuracion arquitectonica y posiciones de cuadros en una base de datos SQLite.



Estructura del Proyecto

```
museo_virtual_backend/
├─ server.js           # Servidor Express principal
├─ database.js         # Configuracion SQLite + helpers
├─ package.json        # Dependencias
├─ museo.db            # Base de datos (se crea automaticamente)
├─ routes/
│   └─ museums.js      # API REST completa
├─ uploads/
│   ├── images/        # Imagenes subidas (cuadros)
│   └─ audio/          # Archivos de audio (ambiente)
└─ public/
    └─ index.html       # Frontend conectado al backend
```



Instalacion y Uso

1. Requisitos

- **Node.js** v16+ (descargar de <https://nodejs.org>)
- npm (viene incluido con Node.js)

2. Instalar dependencias

```
cd museo_virtual_backend
npm install
```

3. Iniciar el servidor

```
npm start
```

El servidor corra en <http://localhost:3000>.

Para desarrollo con auto-recarga:

```
npm run dev
```

4. Abrir el museo

Abre tu navegador en:

```
http://localhost:3000
```



API Endpoints

Metodo	Endpoint	Descripcion
GET	/api/health	Verificar estado del servidor
POST	/api/museums	Crear nuevo museo
GET	/api/museums	Listar todos los museos
GET	/api/museums/:id	Obtener museo completo (config + archivos)
PUT	/api/museums/:id	Renombrar museo
DELETE	/api/museums/:id	Eliminar museo y todo su contenido
POST	/api/museums/:id/images	Subir imagen (cuadro)
GET	/api/museums/:id/images	Listar imagenes del museo
PUT	/api/museums/:id/images/:imgId	Actualizar posicion de imagen
DELETE	/api/museums/:id/images/:imgId	Eliminar imagen
POST	/api/museums/:id/audio	Subir audio (ambiente)
GET	/api/museums/:id/audio	Obtener audio del museo
DELETE	/api/museums/:id/ audio/:audioId	Eliminar audio
PUT	/api/museums/:id/config	Guardar configuracion arquitectonica
GET	/api/museums/:id/config	Cargar configuracion arquitectonica

🎨 Funcionalidades del Frontend (conectado)

1. **Crear Museo:** Escribe un nombre y haz click en “Crear”
2. **Cargar Museo:** Selecciona de la lista o accede directo via URL: `http://localhost:3000/?museum=ID_DEL_MUSEO`
3. **Guardar Configuracion:** Persiste materiales de piso/pared/techo, puertas y ventanas
4. **Subir Imagen:** Se envia al servidor y se carga automaticamente en la escena 3D
5. **Arrastrar Cuadros:** Al soltar un cuadro, su posicion se guarda en el servidor automaticamente
6. **Subir Audio:** Se almacena en el servidor y reproduce desde alli
7. **Compartir:** Copia la URL con el parametro `?museum=ID` para que otros vean tu museo

🗄 Esquema de Base de Datos (SQLite)

Tabla **museums**

- **id** (TEXT PK): UUID del museo
- **name** (TEXT): Nombre del museo
- **created_at, updated_at** (DATETIME)

Tabla **images**

- **id** (INTEGER PK): ID autoincremental
- **museum_id** (TEXT FK): Museo al que pertenece
- **filename** (TEXT): Nombre del archivo en disco
- **original_name** (TEXT): Nombre original del archivo
- **position_x/y/z** (REAL): Posicion 3D en la escena
- **rotation_y** (REAL): Rotacion en el eje Y
- **wall_target** (TEXT): Pared donde esta colgado

Tabla **audio**

- `id` (INTEGER PK)
- `museum_id` (TEXT FK)
- `filename`, `original_name`, `mime_type`, `size`
- `volume`, `is_looping`

Tabla **configurations**

- `id` (INTEGER PK)
- `museum_id` (TEXT FK UNIQUE)
- `floor_material`, `wall_material`, `ceiling_material` (TEXT)
- `doors_json`, `windows_json` (TEXT): Arrays JSON con datos de puertas/ventanas

⚠ **Notas Importantes**

- **CORS:** El backend ya incluye cors habilitado. Si despliegas en produccion, configura los origenes permitidos.
- **Almacenamiento:** Las imagenes y audio se guardan en `uploads/`. En produccion considera usar un servicio de almacenamiento en la nube (AWS S3, Cloudinary, etc.).
- **SQLite:** Es una base de datos basada en archivos. Para produccion con alta concurrencia, considera migrar a PostgreSQL o MySQL.
- **Tamano de archivo:** Limite de 10MB para imagenes y 50MB para audio.

🔧 **Despliegue en Produccion (ej: Railway, Render, VPS)**

1. Sube todo el proyecto a tu servidor
2. Ejecuta `npm install`
3. Configura una variable de entorno `PORT` si es necesario
4. Ejecuta `npm start`
5. Asegurate de que las carpetas `uploads/` tengan permisos de escritura

📄 **Licencia**

MIT